

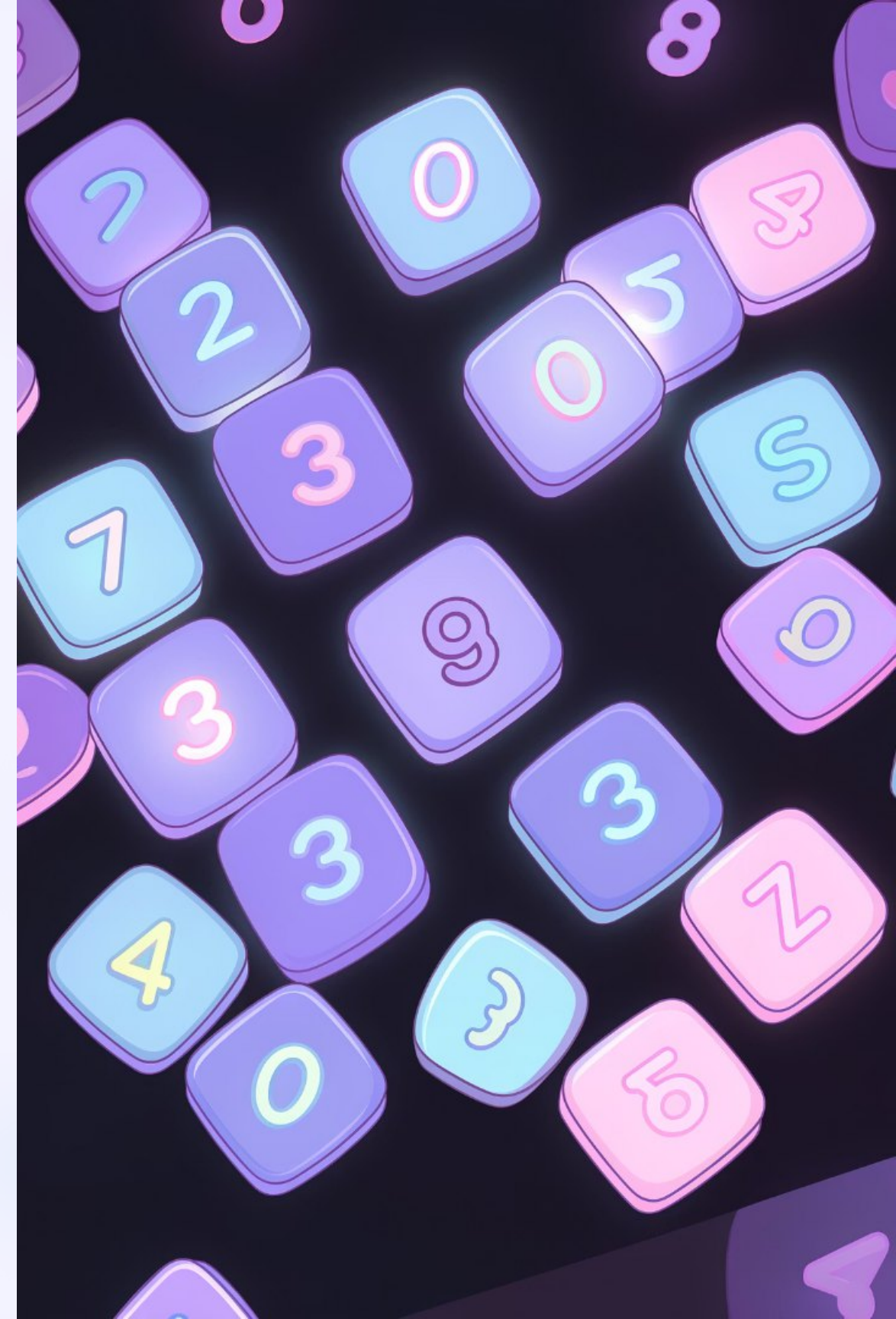
Number Puzzle Game with Scoreboard

A beginner-friendly C console game featuring random number guessing, attempt tracking, and persistent file-based scoreboard — built with clean modular design.

C PROGRAMMING

FILE I/O

CONSOLE GAME



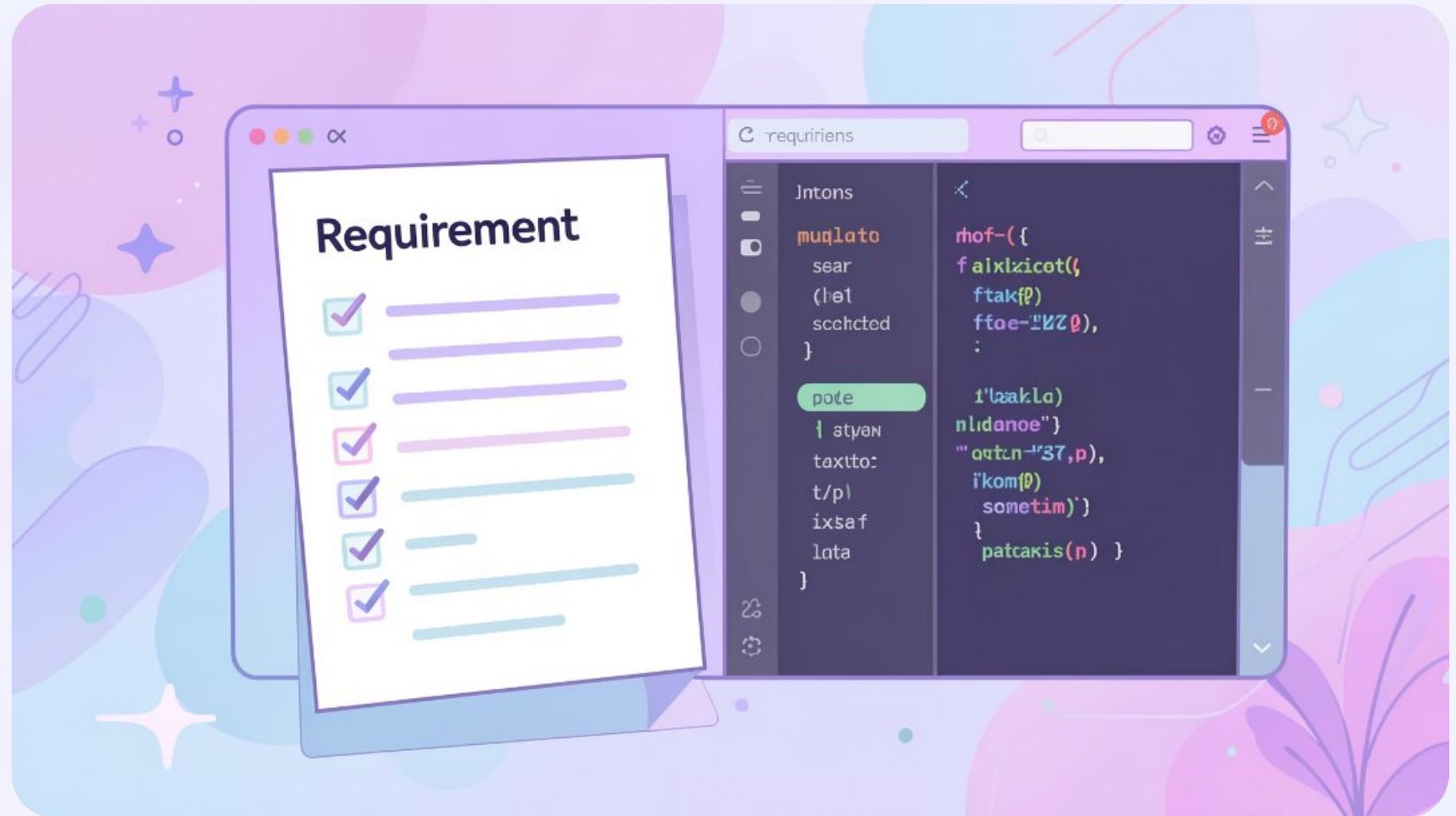
Project Requirements

Functional Requirements

- Generate a random number between 1 and 100
- Accept and validate player guesses in a loop
- Track number of attempts per game session
- Save player name and score to a file
- Read and display all saved scores

Non-Functional Requirements

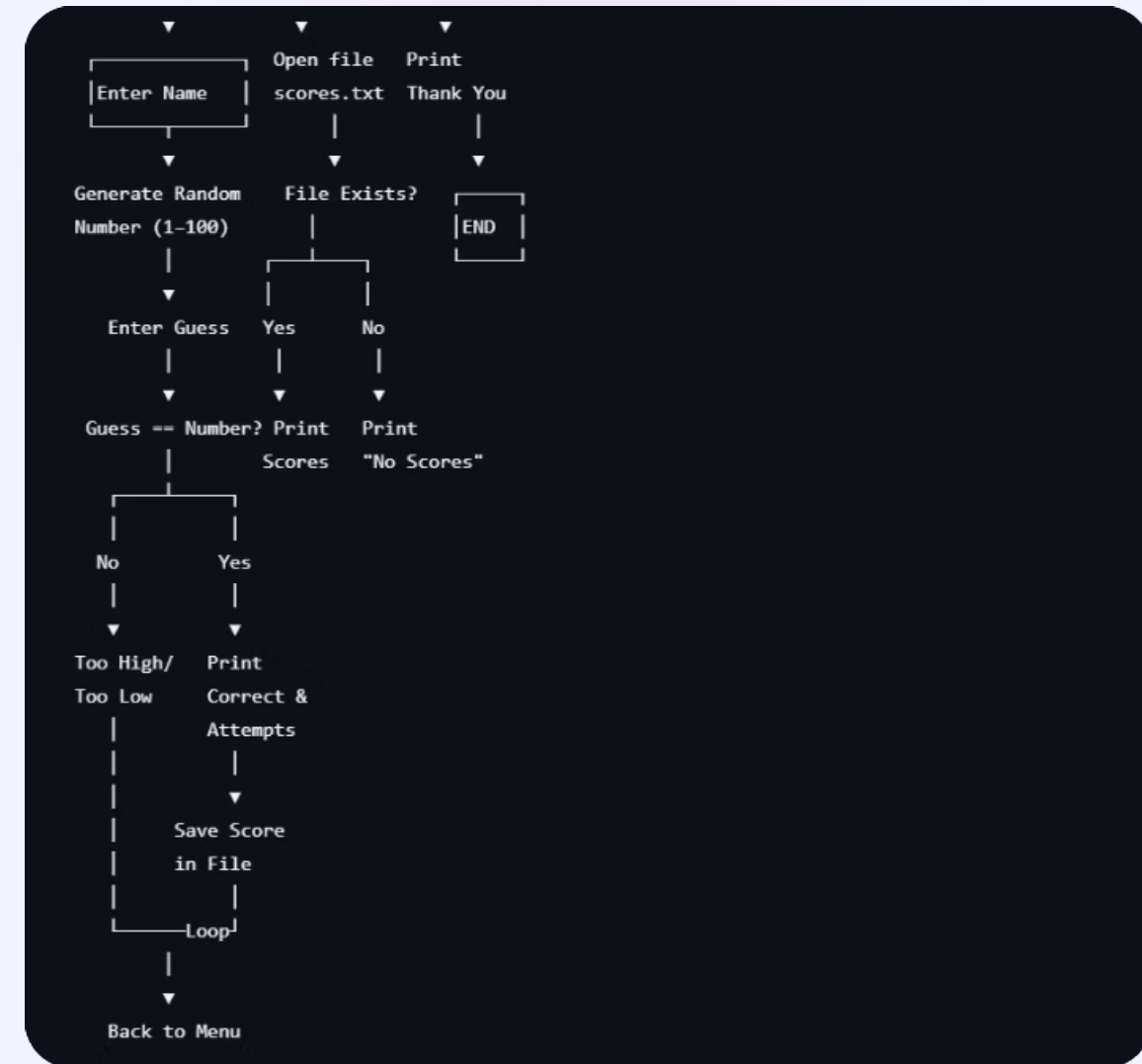
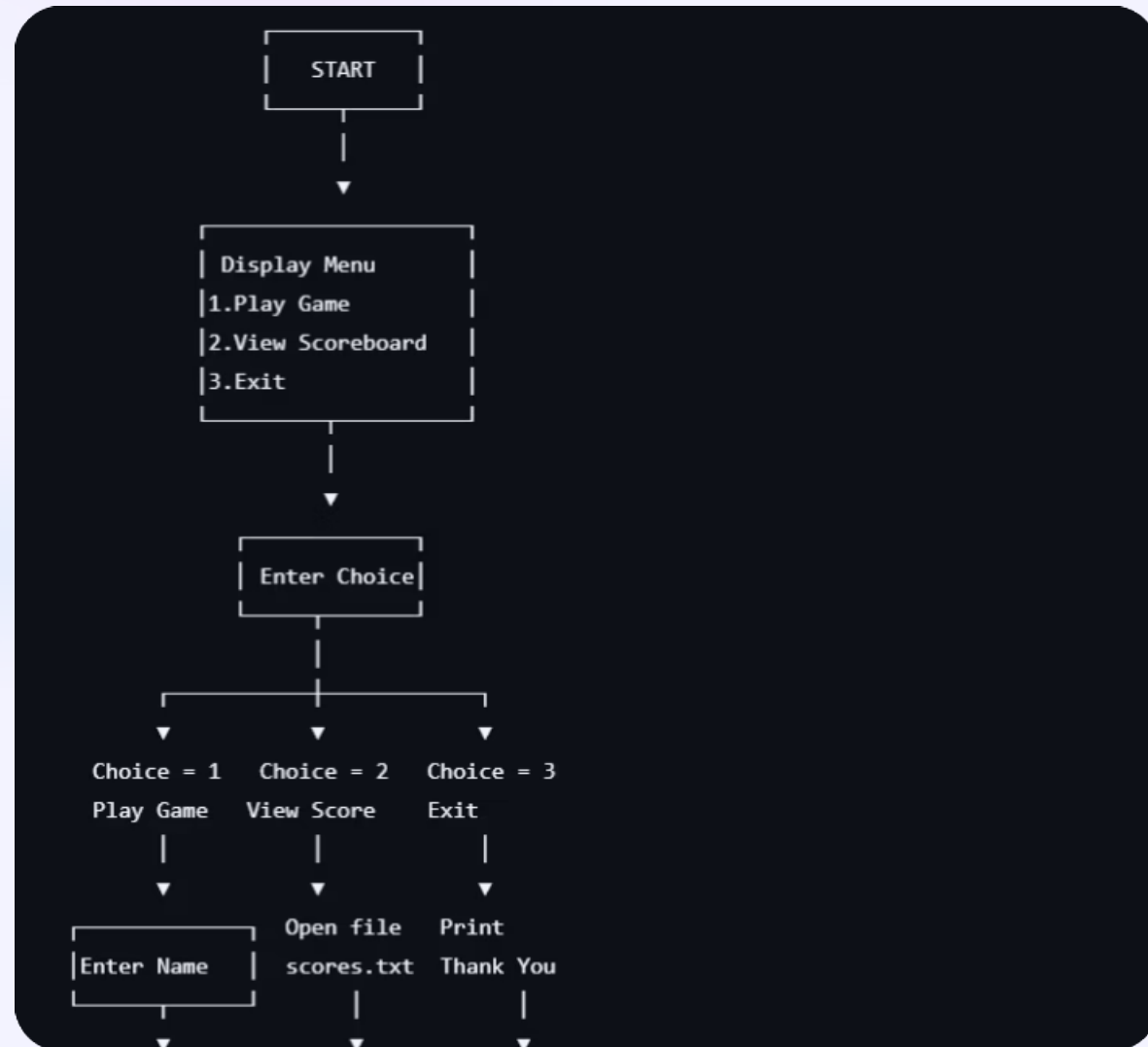
- Simple text-based console interface
- Persistent storage using scores.txt
- Error handling for file operations
- Modular code with separate functions



ALGORITHM:

```
Step 1: Start the program
Step 2: Display the menu:
->Play Game
->View Scoreboard
->Exit
Step 3: Read the user's choice.
Step 4: Check the choice.
Step 5: If choice = 1, perform the following:
->Ask the user to enter their name
->Generate a random number between 1 and 100
->Set attempts = 0
Step 6: Ask the user to enter a guess number
Step 7: Increase the attempt counter by 1
Step 8: Compare the guessed number with the random number
->If guess is greater than the number, display "Too High"
->If guess is smaller than the number, display "Too Low"
->If guess equals the number, display "Correct! Number guessed"
Step 9: Repeat Steps 6-8 until the correct number is guessed
Step 10: Open the file scores.txt in append mode
Step 11: Save the player name and number of attempts in the file
Step 12: Close the file
Step 13: Return to the main menu
Step 14: If choice = 2:
->Open scores.txt in read mode
->Display all saved scores (name and attempts)
Close the file
Step 15: If choice = 3:
->Display "Thank you for playing!"
Step 16: Stop the program
```

FLOW CHART:



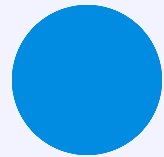
Program Modules

The program is organized into 3 core modules, each handling a distinct responsibility.



Module 1: main()

Displays the main menu in a loop.
Routes user input to `playGame()` or `viewScoreboard()` using a switch statement.



Module 2: playGame()

Handles the full game session —
reads player name, generates
random number, runs the guess
loop, counts attempts, and writes
score to file.



Module 3: viewScoreboard()

Opens `scores.txt` in read mode and
prints all saved player names and
attempt counts. Gracefully handles
missing file.

Algorithm Complexity & Mathematics

Worst-Case Analysis

In the worst case (random guessing), the player may need up to 100 attempts. With an optimal binary search strategy, the maximum attempts needed is:

$$\lceil \log_2(100) \rceil = 7 \text{ attempts}$$

This is because each optimal guess eliminates half the remaining search space, following a logarithmic pattern.

Average Case

For random guessing on a uniform distribution over [1, 100], the expected number of attempts is 50. With binary search, the average is approximately ~5.7 attempts.

**RANDOM
GUESSING.**
Worst case:
100 attempts.
Average: 50.
No strategy.
**Uses accent
color.**



**BINARY
SEARCH.**
Worst case:
7 attempts.
Average: 5.7.
**Halves
search each
Uses
secondary
accents.**



ABCDEFGHIJKLMNOPQRSTUVWXYZ
abcdefghijklmnopqrstuvwxyz
0123456789.
This is the accent color, use it
for highlights and decorations

File I/O: Saving Scores

Scores are appended to `scores.txt` after every game session using standard C file operations.

01

Open File (Append Mode)

`fopen("scores.txt", "a")` – creates the file if it doesn't exist; appends if it does.

03

Close & Error Check

`fclose(fp)` flushes the buffer. If `fopen` returns `NULL`, an error message is displayed.

02

Write Record

`fprintf(fp, "%s %d\n", name, attempts)` – writes the player name and attempt count as one line.



Reading the Scoreboard



viewScoreboard() Logic

The function opens `scores.txt` in read mode and uses a `while` loop with `fscanf` to parse each line until EOF.

i Each record in the file stores one player per line: `PlayerName`
`Attempts`

Error Handling

If the file doesn't exist yet (`fp == NULL`), the function prints "No scores available yet" and returns gracefully — no crash.

Key C Concepts Used

Random Numbers

`srand()` + `rand()` from `<stdlib.h>` and `<time.h>` for non-repeating sequences each run.

File I/O

`fopen`, `fprintf`, `fscanf`, `fclose` for persistent read/write operations on text files.

do-while Loop

Guarantees at least one guess is made before checking the exit condition — ideal for input loops.

Modular Functions

Separating logic into `main()`, `playGame()`, and `viewScoreboard()` improves readability and reuse.



Summary & Next Steps

What You Built

A fully functional C console game with 3 modules, random number logic, attempt tracking, and persistent file-based scoreboard.

Extend It

Sort the scoreboard by fewest attempts, add difficulty levels (range 1–50 vs 1–500), or display a top-3 leaderboard.

Key Takeaway

File I/O with `fopen/fprintf/fscanf` is the foundation of persistence in C – master it and you can store any game data.